

哈希

两数之和

思路：一边遍历，一边用哈希表记录“值 -> 下标”。当前数字是 x ，如果之前出现过 $target - x$ ，直接返回对应下标。

```
1 function twoSum(nums, target) {
2   const map = new Map()
3
4   for (let i = 0; i < nums.length; i++) {
5     const need = target - nums[i]
6     if (map.has(need)) {
7       return [map.get(need), i]
8     }
9     map.set(nums[i], i)
10  }
11
12  return []
13 }
```

时间复杂度 $O(n)$ ，空间复杂度 $O(n)$ 。

字母异位词分组

思路：异位词排序后字符串相同，把排序后的结果作为 key 对字符串进行分组。

```
1 function groupAnagrams(strs) {
2   const map = new Map()
3
4   for (const str of strs) {
5     const key = str.split('').sort().join('')
6     if (!map.has(key)) map.set(key, [])
7     map.get(key).push(str)
8   }
9
10  return [...map.values()]
11 }
```

时间复杂度近似 $O(n * k \log k)$ ， k 是字符串平均长度。

最长连续序列

思路：先将数组元素放入 Set 便于快速查询；只从“连续序列起点”（即 $num - 1$ 不存在于 Set 中）开始扩展，避免重复遍历。

```
1 function longestConsecutive(nums) {
2   const set = new Set(nums)
3   let ans = 0
```

```

4
5   for (const num of set) {
6       if (!set.has(num - 1)) {
7           let cur = num
8           let len = 1
9           while (set.has(cur + 1)) {
10              cur++
11              len++
12          }
13          ans = Math.max(ans, len)
14      }
15  }
16
17  return ans
18 }

```

时间复杂度 $O(n)$ 。

双指针

移动零

思路：快指针遍历寻找非零元素，慢指针负责放置非零元素；遍历结束后，将慢指针之后的位置全部补 0，实现原地操作。

```

1  function moveZeroes(nums) {
2      let slow = 0
3
4      for (let fast = 0; fast < nums.length; fast++) {
5          if (nums[fast] !== 0) {
6              [nums[slow], nums[fast]] = [nums[fast], nums[slow]]
7              slow++
8          }
9      }
10
11     return nums
12 }

```

时间复杂度 $O(n)$ ，原地操作（空间复杂度 $O(1)$ ）。

盛最多水的容器

思路：左右双指针分别指向数组两端，容器面积取决于短板（较短的边）和两指针间距；每次移动较短的一边，逐步缩小范围，更新最大面积。

```

1  function maxArea(height) {
2      let left = 0
3      let right = height.length - 1
4      let ans = 0

```

```

5
6   while (left < right) {
7       const area = Math.min(height[left], height[right]) * (right - left)
8       ans = Math.max(ans, area)
9
10      if (height[left] < height[right]) {
11          left++
12      } else {
13          right--
14      }
15  }
16
17  return ans
18 }

```

时间复杂度 $O(n)$ 。

三数之和

思路：先对数组排序，固定一个元素作为基准，剩下两个元素用双指针寻找；注意去重（基准元素去重、双指针元素去重），避免重复结果。

```

1  function threeSum(nums) {
2      nums.sort((a, b) => a - b)
3      const res = []
4
5      for (let i = 0; i < nums.length - 2; i++) {
6          if (i > 0 && nums[i] === nums[i - 1]) continue
7          if (nums[i] > 0) break // 基准元素大于0，三数和必大于0，直接退出
8
9          let left = i + 1
10         let right = nums.length - 1
11
12         while (left < right) {
13             const sum = nums[i] + nums[left] + nums[right]
14
15             if (sum === 0) {
16                 res.push([nums[i], nums[left], nums[right]])
17                 // 双指针去重
18                 while (left < right && nums[left] === nums[left + 1]) left++
19                 while (left < right && nums[right] === nums[right - 1]) right--
20                 left++
21                 right--
22             } else if (sum < 0) {
23                 left++ // 和偏小，移动左指针增大数值
24             } else {
25                 right-- // 和偏大，移动右指针减小数值
26             }
27         }
28     }
29 }

```

```

27     }
28 }
29
30 return res
31 }

```

时间复杂度 $O(n^2)$ 。

接雨水

思路：双指针维护左侧最大值和右侧最大值；哪边的最大值更小，就结算哪边的雨水量（当前位置雨水量 = 对应侧最大值 - 当前位置高度），逐步缩小范围。

```

1 function trap(height) {
2     let left = 0
3     let right = height.length - 1
4     let leftMax = 0
5     let rightMax = 0
6     let ans = 0
7
8     while (left < right) {
9         leftMax = Math.max(leftMax, height[left])
10        rightMax = Math.max(rightMax, height[right])
11
12        if (leftMax < rightMax) {
13            ans += leftMax - height[left]
14            left++
15        } else {
16            ans += rightMax - height[right]
17            right--
18        }
19    }
20
21    return ans
22 }

```

除自身以外数组的乘积

思路：分两次遍历，第一次计算每个元素左侧所有元素的乘积，第二次计算每个元素右侧所有元素的乘积，两者相乘即为结果。

```

1 function productExceptSelf(nums) {
2     const n = nums.length
3     const res = Array(n).fill(1)
4
5     let pre = 1
6     for (let i = 0; i < n; i++) {
7         res[i] = pre
8         pre *= nums[i]
9     }
10
11    for (let i = n - 1; i >= 0; i--) {
12        res[i] *= pre
13        pre *= nums[i]
14    }
15
16    return res
17 }

```

```

9     }
10
11     let suf = 1
12     for (let i = n - 1; i >= 0; i--) {
13         res[i] *= suf
14         suf *= nums[i]
15     }
16
17     return res
18 }

```

时间复杂度 $O(n)$ ，额外空间忽略结果数组时为 $O(1)$ 。

缺失的第一个正数

思路：原地哈希，将值为 x 的元素尽量放到下标 $x - 1$ 的位置（因为正数范围是 $1 \sim n$ ，对应下标 $0 \sim n-1$ ）；遍历数组，找到第一个下标与值不匹配的位置，其下标+1即为缺失的最小正数。

```

1 function firstMissingPositive(nums) {
2     const n = nums.length
3
4     for (let i = 0; i < n; i++) {
5         while (
6             nums[i] >= 1 &&
7             nums[i] <= n &&
8             nums[nums[i] - 1] !== nums[i]
9         ) {
10             [nums[i], nums[nums[i] - 1]] = [nums[nums[i] - 1], nums[i]]
11         }
12     }
13
14     for (let i = 0; i < n; i++) {
15         if (nums[i] !== i + 1) return i + 1
16     }
17
18     return n + 1
19 }

```

难点在于“原地把正确值放到正确位置”，避免额外空间消耗。

矩阵

矩阵置零

思路：用矩阵的第一行和第一列作为标记位，记录对应行/列是否需要置零；额外用两个变量记录第一行、第一列本身是否包含 0，避免标记位污染原数据。

```

1 function setZeroes(matrix) {
2     const m = matrix.length

```

```
3   const n = matrix[0].length
4   let row0 = false // 标记第一行是否有0
5   let col0 = false // 标记第一列是否有0
6
7   // 检查第一列是否有0
8   for (let i = 0; i < m; i++) {
9       if (matrix[i][0] === 0) col0 = true
10  }
11
12  // 检查第一行是否有0
13  for (let j = 0; j < n; j++) {
14      if (matrix[0][j] === 0) row0 = true
15  }
16
17  // 用第一行和第一列标记其他行/列是否有0
18  for (let i = 1; i < m; i++) {
19      for (let j = 1; j < n; j++) {
20          if (matrix[i][j] === 0) {
21              matrix[i][0] = 0
22              matrix[0][j] = 0
23          }
24      }
25  }
26
27  // 根据标记位置零
28  for (let i = 1; i < m; i++) {
29      for (let j = 1; j < n; j++) {
30          if (matrix[i][0] === 0 || matrix[0][j] === 0) {
31              matrix[i][j] = 0
32          }
33      }
34  }
35
36  // 处理第一行
37  if (row0) {
38      for (let j = 0; j < n; j++) matrix[0][j] = 0
39  }
40
41  // 处理第一列
42  if (col0) {
43      for (let i = 0; i < m; i++) matrix[i][0] = 0
44  }
45 }
```

螺旋矩阵

思路：维护上下左右四个边界，按“上→右→下→左”的顺序遍历矩阵，遍历完一行/一列后收缩对应边界，直到边界交叉。

```
1 function spiralOrder(matrix) {
2     const res = []
3     let top = 0
4     let bottom = matrix.length - 1
5     let left = 0
6     let right = matrix[0].length - 1
7
8     while (top <= bottom && left <= right) {
9         // 上：从左到右
10        for (let j = left; j <= right; j++) res.push(matrix[top][j])
11        top++
12
13        // 右：从上到下
14        for (let i = top; i <= bottom; i++) res.push(matrix[i][right])
15        right--
16
17        // 下：从右到左（需判断top <= bottom，避免重复遍历）
18        if (top <= bottom) {
19            for (let j = right; j >= left; j--) res.push(matrix[bottom][j])
20            bottom--
21        }
22
23        // 左：从下到上（需判断left <= right，避免重复遍历）
24        if (left <= right) {
25            for (let i = bottom; i >= top; i--) res.push(matrix[i][left])
26            left++
27        }
28    }
29
30    return res
31 }
```

旋转图像

思路：分两步操作：先将矩阵转置（行变列、列变行），再将转置后的矩阵每行进行翻转，即可得到顺时针旋转90度的结果。

```
1 function rotate(matrix) {
2     const n = matrix.length
3
4     // 第一步：转置矩阵
5     for (let i = 0; i < n; i++) {
6         for (let j = i + 1; j < n; j++) {
```

```

7      ;[matrix[i][j], matrix[j][i]] = [matrix[j][i], matrix[i][j]]
8    }
9  }
10
11  // 第二步：翻转每行
12  for (const row of matrix) {
13    row.reverse()
14  }
15 }

```

搜索二维矩阵 II

思路：从矩阵右上角开始遍历，利用矩阵“每行递增、每列递增”的特性；比目标值大则左移，比目标值小则下移，高效缩小查找范围。

```

1  function searchMatrix(matrix, target) {
2    let i = 0
3    let j = matrix[0].length - 1
4
5    while (i < matrix.length && j >= 0) {
6      if (matrix[i][j] === target) return true
7      if (matrix[i][j] > target) j--
8      else i++
9    }
10
11    return false
12  }

```

时间复杂度 $O(m + n)$ ， m 为矩阵行数， n 为矩阵列数。

链表

相交链表

思路：双指针分别从两个链表头出发，走完自己的链表后切换到对方链表继续走；由于两指针总路程相同，若链表相交，必在交点相遇；若不相交，最终同时指向 `null`。

```

1  function getIntersectionNode(headA, headB) {
2    let p1 = headA
3    let p2 = headB
4
5    while (p1 !== p2) {
6      p1 = p1 ? p1.next : headB
7      p2 = p2 ? p2.next : headA
8    }
9
10   return p1
11 }

```

反转链表

```
1 function reverseList(head) {
2   let prev = null
3   let cur = head
4
5   while (cur) {
6     const next = cur.next // 保存下一个节点
7     cur.next = prev // 反转当前节点指针
8     prev = cur // 移动prev到当前节点
9     cur = next // 移动cur到下一个节点
10  }
11
12  return prev // 反转后prev为新表头
13 }
```

回文链表

思路：用快慢指针找到链表 midpoint，反转后半段链表，再将前半段与反转后的后半段逐一比较，若所有节点值相等则为回文链表。

```
1 function isPalindrome(head) {
2   let slow = head
3   let fast = head
4
5   // 快慢指针找中点 (slow最终指向后半段起点)
6   while (fast && fast.next) {
7     slow = slow.next
8     fast = fast.next.next
9   }
10
11  // 反转后半段链表
12  let prev = null
13  while (slow) {
14    const next = slow.next
15    slow.next = prev
16    prev = slow
17    slow = next
18  }
19
20  // 比较前半段和反转后的后半段
21  let p1 = head
22  let p2 = prev
23  while (p2) {
24    if (p1.val !== p2.val) return false
25    p1 = p1.next
26    p2 = p2.next
27  }
```

```
28
29     return true
30 }
```

环形链表

思路：快慢指针法，慢指针每次走1步，快指针每次走2步；若链表有环，快慢指针必相遇；若快指针先到达null，则无环。

```
1  function hasCycle(head) {
2      let slow = head
3      let fast = head
4
5      while (fast && fast.next) {
6          slow = slow.next
7          fast = fast.next.next
8          if (slow === fast) return true
9      }
10
11     return false
12 }
```

环形链表 II

思路：第一步用快慢指针找到相遇点；第二步将一个指针移回表头，两个指针同时每次走1步，再次相遇的位置即为环的入口。

```
1  function detectCycle(head) {
2      let slow = head
3      let fast = head
4
5      // 第一步：找到快慢指针相遇点
6      while (fast && fast.next) {
7          slow = slow.next
8          fast = fast.next.next
9
10         if (slow === fast) {
11             // 第二步：找环入口
12             let p = head
13             while (p !== slow) {
14                 p = p.next
15                 slow = slow.next
16             }
17             return p
18         }
19     }
20
21     return null
22 }
```

合并两个有序链表

思路：用虚拟头节点（dummy）简化边界处理，双指针分别遍历两个链表，每次将较小的节点接入结果链表，最后接入剩余节点。

```
1 function mergeTwoLists(list1, list2) {
2   const dummy = new ListNode(0)
3   let cur = dummy
4
5   while (list1 && list2) {
6     if (list1.val < list2.val) {
7       cur.next = list1
8       list1 = list1.next
9     } else {
10      cur.next = list2
11      list2 = list2.next
12    }
13    cur = cur.next
14  }
15
16  // 接入剩余节点
17  cur.next = list1 || list2
18  return dummy.next
19 }
```

两数相加

思路：用虚拟头节点存储结果，遍历两个链表，逐位相加，记录进位；注意处理链表长度不一致和最后一位进位的情况。

```
1 function addTwoNumbers(l1, l2) {
2   const dummy = new ListNode(0)
3   let cur = dummy
4   let carry = 0 // 进位
5
6   while (l1 || l2 || carry) {
7     const sum = (l1 ? l1.val : 0) + (l2 ? l2.val : 0) + carry
8     carry = Math.floor(sum / 10) // 更新进位
9     cur.next = new ListNode(sum % 10) // 当前位结果
10    cur = cur.next
11
12    // 移动链表指针
13    if (l1) l1 = l1.next
14    if (l2) l2 = l2.next
15  }
16 }
```

```
17     return dummy.next
18 }
```

删除链表的倒数第 N 个结点

思路：双指针法，快指针先进 N 步，然后快慢指针同时前进，当快指针到达链表末尾时，慢指针指向的就是倒数第 N 个节点的前驱节点，直接删除目标节点。

```
1  function removeNthFromEnd(head, n) {
2      const dummy = new ListNode(0, head) // 虚拟头节点，避免处理头节点删除
3      let fast = dummy
4      let slow = dummy
5
6      // 快指针先进n步
7      for (let i = 0; i < n; i++) fast = fast.next
8
9      // 快慢指针同时前进，直到快指针到达末尾
10     while (fast.next) {
11         fast = fast.next
12         slow = slow.next
13     }
14
15     // 删除倒数第n个节点
16     slow.next = slow.next.next
17     return dummy.next
18 }
```

两两交换链表中的节点

思路：用虚拟头节点简化操作，遍历链表，每次交换相邻两个节点，注意保存节点指针，避免链表断裂。

```
1  function swapPairs(head) {
2      const dummy = new ListNode(0, head)
3      let cur = dummy
4
5      // 确保当前节点后有两个节点可交换
6      while (cur.next && cur.next.next) {
7          const a = cur.next
8          const b = cur.next.next
9
10         // 交换a和b
11         a.next = b.next
12         b.next = a
13         cur.next = b
14
15         // 移动cur到下一组待交换节点的前驱
16         cur = a
```

```
17     }
18
19     return dummy.next
20 }
```

K 个一组翻转链表

思路：用虚拟头节点，先判断当前剩余节点是否足够 K 个，足够则翻转这 K 个节点，否则直接返回；翻转后连接前后链表，重复操作直到遍历完成。

```
1 function reverseKGroup(head, k) {
2     const dummy = new ListNode(0, head)
3     let pre = dummy // 每K个节点的前驱节点
4
5     while (true) {
6         let tail = pre
7         // 检查剩余节点是否足够k个
8         for (let i = 0; i < k; i++) {
9             tail = tail.next
10            if (!tail) return dummy.next
11        }
12
13        const next = tail.next // 保存下一组节点的头
14        // 翻转当前k个节点，返回新的头和尾
15        let [newHead, newTail] = reverse(pre.next, tail)
16
17        // 连接翻转后的节点
18        pre.next = newHead
19        newTail.next = next
20        pre = newTail // 更新pre为下一组的前驱
21    }
22 }
23
24 // 辅助函数：翻转从head到tail的链表，返回新的头和尾
25 function reverse(head, tail) {
26     let prev = tail.next
27     let cur = head
28
29     while (prev !== tail) {
30         const next = cur.next
31         cur.next = prev
32         prev = cur
33         cur = next
34     }
35
36     return [tail, head]
37 }
```

随机链表的复制

思路：用哈希表记录“旧节点 -> 新节点”的映射，先遍历链表创建所有新节点并存储映射，再遍历一次链表，完善新节点的 next 和 random 指针。

```
1 function copyRandomList(head) {
2   if (!head) return null
3
4   const map = new Map()
5   let cur = head
6
7   // 第一步：创建新节点，存储旧节点与新节点的映射
8   while (cur) {
9     map.set(cur, new Node(cur.val))
10    cur = cur.next
11  }
12
13  // 第二步：完善新节点的next和random指针
14  cur = head
15  while (cur) {
16    map.get(cur).next = cur.next ? map.get(cur.next) : null
17    map.get(cur).random = cur.random ? map.get(cur.random) : null
18    cur = cur.next
19  }
20
21  return map.get(head)
22 }
```

排序链表

思路：链表适合用归并排序，避免数组排序的额外空间消耗；先拆分链表为两半，分别排序，再合并两个有序链表，递归执行直到链表长度为1。

```
1 function sortList(head) {
2   // 终止条件：链表为空或只有一个节点
3   if (!head || !head.next) return head
4
5   // 快慢指针找中点，拆分链表
6   let slow = head
7   let fast = head.next
8   while (fast && fast.next) {
9     slow = slow.next
10    fast = fast.next.next
11  }
12  const mid = slow.next
13  slow.next = null // 拆分左半部分和右半部分
14
15  // 递归排序左右两半
```

```

16     const left = sortList(head)
17     const right = sortList(mid)
18
19     // 合并两个有序链表
20     return merge(left, right)
21 }
22
23 // 辅助函数：合并两个有序链表
24 function merge(l1, l2) {
25     const dummy = new ListNode(0)
26     let cur = dummy
27
28     while (l1 && l2) {
29         if (l1.val < l2.val) {
30             cur.next = l1
31             l1 = l1.next
32         } else {
33             cur.next = l2
34             l2 = l2.next
35         }
36         cur = cur.next
37     }
38
39     cur.next = l1 || l2
40     return dummy.next
41 }

```

合并 K 个升序链表

思路：分治合并，将 K 个链表两两合并，逐步减少链表数量，最终合并为一个有序链表；相比逐个合并，分治可降低时间复杂度。

```

1  function mergeKLists(lists) {
2      if (!lists.length) return null
3
4      let interval = 1
5      // 分治合并：每次合并interval和interval+interval位置的链表
6      while (interval < lists.length) {
7          for (let i = 0; i + interval < lists.length; i += interval * 2) {
8              lists[i] = mergeTwo(lists[i], lists[i + interval])
9          }
10         interval *= 2
11     }
12
13     return lists[0]
14 }
15
16 // 辅助函数：合并两个有序链表

```

```

17 function mergeTwo(a, b) {
18     const dummy = new ListNode(0)
19     let cur = dummy
20
21     while (a && b) {
22         if (a.val < b.val) {
23             cur.next = a
24             a = a.next
25         } else {
26             cur.next = b
27             b = b.next
28         }
29         cur = cur.next
30     }
31
32     cur.next = a || b
33     return dummy.next
34 }

```

LRU 缓存

思路：用“哈希表 + 双向链表”实现；哈希表负责 $O(1)$ 查找，双向链表负责 $O(1)$ 插入和删除，维护最近使用顺序（最近使用的节点移到表头，满容量时删除表尾节点）。

```

1 class Node {
2     constructor(key, val) {
3         this.key = key
4         this.val = val
5         this.prev = null
6         this.next = null
7     }
8 }
9
10 class LRUCache {
11     constructor(capacity) {
12         this.capacity = capacity
13         this.map = new Map() // 存储key->node
14         // 虚拟头和虚拟尾，简化边界操作
15         this.head = new Node(0, 0)
16         this.tail = new Node(0, 0)
17         this.head.next = this.tail
18         this.tail.prev = this.head
19     }
20
21     // 移除节点
22     remove(node) {
23         node.prev.next = node.next
24         node.next.prev = node.prev

```

```
25     }
26
27     // 将节点插入到表头（最近使用）
28     insert(node) {
29         node.next = this.head.next
30         node.prev = this.head
31         this.head.next.prev = node
32         this.head.next = node
33     }
34
35     // 获取值
36     get(key) {
37         if (!this.map.has(key)) return -1
38         const node = this.map.get(key)
39         this.remove(node) // 移除当前节点
40         this.insert(node) // 移到表头，标记为最近使用
41         return node.val
42     }
43
44     // 存入值
45     put(key, value) {
46         // 若已存在，先移除
47         if (this.map.has(key)) {
48             this.remove(this.map.get(key))
49         }
50
51         // 创建新节点并插入表头
52         const node = new Node(key, value)
53         this.insert(node)
54         this.map.set(key, node)
55
56         // 超过容量，删除表尾节点（最久未使用）
57         if (this.map.size > this.capacity) {
58             const del = this.tail.prev
59             this.remove(del)
60             this.map.delete(del.key)
61         }
62     }
63 }
```

二叉树

二叉树的中序遍历

```
1 function inorderTraversal(root) {
2   const res = []
3
4   // 递归辅助函数
5   function dfs(node) {
6     if (!node) return
7     dfs(node.left) // 左
8     res.push(node.val) // 中
9     dfs(node.right) // 右
10  }
11
12  dfs(root)
13  return res
14 }
```

二叉树的最大深度

思路：递归遍历，树的最大深度 = 左子树最大深度和右子树最大深度的最大值 + 1（当前节点）。

```
1 function maxDepth(root) {
2   if (!root) return 0
3   return Math.max(maxDepth(root.left), maxDepth(root.right)) + 1
4 }
```

翻转二叉树

思路：递归遍历，交换当前节点的左右子树，再递归翻转左右子树。

```
1 function invertTree(root) {
2   if (!root) return null
3   ;[root.left, root.right] = [root.right, root.left] // 交换左右子树
4   invertTree(root.left)
5   invertTree(root.right)
6   return root
7 }
```

对称二叉树

思路：递归检查左右子树是否对称，即左子树的左节点与右子树的右节点相等，左子树的右节点与右子树的左节点相等。

```

1 function isSymmetric(root) {
2   // 辅助函数：检查两个节点是否对称
3   function check(a, b) {
4     if (!a && !b) return true // 都为空，对称
5     if (!a || !b || a.val !== b.val) return false // 一个为空或值不相等，
    不对称
6     // 递归检查：a的左 vs b的右，a的右 vs b的左
7     return check(a.left, b.right) && check(a.right, b.left)
8   }
9
10  return check(root.left, root.right)
11 }

```

二叉树的直径

思路：递归计算每个节点的左右子树深度，节点的直径 = 左子树深度 + 右子树深度；遍历所有节点，记录最大直径。

```

1 function diameterOfBinaryTree(root) {
2   let ans = 0
3
4   // 辅助函数：计算节点深度，同时更新直径
5   function depth(node) {
6     if (!node) return 0
7     const left = depth(node.left)
8     const right = depth(node.right)
9     ans = Math.max(ans, left + right) // 更新最大直径
10    return Math.max(left, right) + 1 // 返回当前节点深度
11  }
12
13  depth(root)
14  return ans
15 }

```

二叉树的层序遍历

思路：用队列实现广度优先搜索（BFS），每次遍历一层的所有节点，将节点值存入结果，再将下一层节点加入队列。

```

1 function levelOrder(root) {
2   if (!root) return []
3   const res = []
4   const queue = [root]
5
6   while (queue.length) {
7     const size = queue.length // 当前层节点数
8     const level = [] // 存储当前层节点值
9

```

```

10     for (let i = 0; i < size; i++) {
11         const node = queue.shift()
12         level.push(node.val)
13         // 将下一层节点加入队列
14         if (node.left) queue.push(node.left)
15         if (node.right) queue.push(node.right)
16     }
17
18     res.push(level)
19 }
20
21 return res
22 }

```

将有序数组转换为二叉搜索树

思路：二叉搜索树的中序遍历是有序数组，因此取数组中点作为根节点，左半部分作为左子树，右半部分作为右子树，递归构建。

```

1 function sortedArrayToBST(nums) {
2     // 辅助函数：构建[left, right]区间的二叉搜索树
3     function build(left, right) {
4         if (left > right) return null
5         const mid = Math.floor((left + right) / 2) // 中点作为根节点
6         const root = new TreeNode(nums[mid])
7         root.left = build(left, mid - 1) // 左子树
8         root.right = build(mid + 1, right) // 右子树
9         return root
10    }
11
12    return build(0, nums.length - 1)
13 }

```

验证二叉搜索树

思路：递归遍历，给每个节点设置上下边界（左子树所有节点小于当前节点，右子树所有节点大于当前节点），检查节点值是否在边界内。

```

1 function isValidBST(root) {
2   // 辅助函数：检查节点是否在[low, high]范围内
3   function dfs(node, low, high) {
4     if (!node) return true
5     if (node.val <= low || node.val >= high) return false
6     // 左子树边界：[-∞, 当前节点值], 右子树边界：[当前节点值, +∞]
7     return dfs(node.left, low, node.val) && dfs(node.right, node.val, high)
8   }
9
10  return dfs(root, -Infinity, Infinity)
11 }

```

二叉搜索树中第 K 小的元素

思路：二叉搜索树的中序遍历是升序序列，因此中序遍历到第 K 个节点时，即为第 K 小的元素；用栈实现非递归中序遍历，提高效率。

```

1 function kthSmallest(root, k) {
2   const stack = []
3   let cur = root
4
5   while (stack.length || cur) {
6     // 遍历到最左节点
7     while (cur) {
8       stack.push(cur)
9       cur = cur.left
10    }
11
12    cur = stack.pop()
13    k--
14    if (k === 0) return cur.val // 第k个节点
15    cur = cur.right
16  }
17 }

```

二叉树的右视图

思路：层序遍历，每次遍历一层时，记录该层的最后一个节点值，即为右视图看到的节点。

```

1 function rightSideView(root) {
2   if (!root) return []
3   const res = []
4   const queue = [root]
5
6   while (queue.length) {
7     const size = queue.length
8     for (let i = 0; i < size; i++) {
9       const node = queue.shift()

```

```

10     // 加入下一层节点
11     if (node.left) queue.push(node.left)
12     if (node.right) queue.push(node.right)
13     // 记录当前层最后一个节点
14     if (i === size - 1) res.push(node.val)
15 }
16 }
17
18 return res
19 }

```

二叉树展开为链表

思路：后序遍历，先将右子树展开为链表，再将左子树展开为链表，最后将左子树链表接到根节点右方，左子树置空。

```

1 function flatten(root) {
2     let prev = null // 记录上一个节点（链表的前驱）
3
4     function dfs(node) {
5         if (!node) return
6         dfs(node.right) // 先处理右子树
7         dfs(node.left) // 再处理左子树
8         // 展开链表
9         node.right = prev
10        node.left = null
11        prev = node
12    }
13
14    dfs(root)
15 }

```

从前序与中序遍历序列构造二叉树

思路：前序遍历的第一个元素是根节点，中序遍历中根节点左侧是左子树，右侧是右子树；用哈希表存储中序遍历的节点位置，快速定位根节点，递归构建左右子树。

```

1 function buildTree(preorder, inorder) {
2     // 哈希表存储中序节点值与下标，便于快速查找
3     const map = new Map()
4     inorder.forEach((v, i) => map.set(v, i))
5
6     // 辅助函数：构建[preL, preR]（前序区间）和[inL, inR]（中序区间）的二叉树
7     function build(preL, preR, inL, inR) {
8         if (preL > preR) return null
9
10        const rootVal = preorder[preL] // 前序第一个元素是根节点

```

```

11     const idx = map.get(rootVal) // 根节点在中序的下标
12     const leftSize = idx - inL // 左子树节点数
13
14     const root = new TreeNode(rootVal)
15     // 构建左子树: 前序[preL+1, preL+leftSize], 中序[inL, idx-1]
16     root.left = build(preL + 1, preL + leftSize, inL, idx - 1)
17     // 构建右子树: 前序[preL+leftSize+1, preR], 中序[idx+1, inR]
18     root.right = build(preL + leftSize + 1, preR, idx + 1, inR)
19     return root
20 }
21
22 return build(0, preorder.length - 1, 0, inorder.length - 1)
23 }

```

路径总和 III

思路：前缀和 + 哈希表，记录从根节点到当前节点的前缀和，若当前前缀和与目标和的差值存在于哈希表中，说明存在对应的路径；递归遍历左右子树，回溯时更新哈希表。

```

1 function pathSum(root, targetSum) {
2     const map = new Map()
3     map.set(0, 1) // 前缀和为0的路径有1条（空路径）
4     let ans = 0
5
6     // 辅助函数：递归计算前缀和
7     function dfs(node, sum) {
8         if (!node) return
9
10        sum += node.val
11        // 查找是否存在前缀和为sum - targetSum的路径
12        ans += map.get(sum - targetSum) || 0
13
14        // 更新哈希表，记录当前前缀和
15        map.set(sum, (map.get(sum) || 0) + 1)
16        // 递归左右子树
17        dfs(node.left, sum)
18        dfs(node.right, sum)
19        // 回溯：删除当前前缀和，避免影响其他路径
20        map.set(sum, map.get(sum) - 1)
21    }
22
23    dfs(root, 0)
24    return ans
25 }

```

二叉树的最近公共祖先

思路：递归遍历，若当前节点是 p 或 q，直接返回当前节点；否则递归查找左右子树，若左右子树都找到目标节点，当前节点即为最近公共祖先；若只有一侧找到，返回找到的节点。

```
1 function lowestCommonAncestor(root, p, q) {
2   // 终止条件：节点为空或找到p/q
3   if (!root || root === p || root === q) return root
4
5   // 递归查找左右子树
6   const left = lowestCommonAncestor(root.left, p, q)
7   const right = lowestCommonAncestor(root.right, p, q)
8
9   // 左右子树都找到，当前节点是公共祖先
10  if (left && right) return root
11  // 只有一侧找到，返回找到的节点
12  return left || right
13 }
```

二叉树中的最大路径和

思路：递归遍历，计算每个节点作为“路径起点”的最大贡献值（左子树最大贡献 + 右子树最大贡献 + 当前节点值）；同时记录全局最大路径和，注意贡献值为负时可舍弃（取0）。

```
1 function maxPathSum(root) {
2   let ans = -Infinity
3
4   // 辅助函数：计算当前节点的最大贡献值
5   function dfs(node) {
6     if (!node) return 0
7
8     // 左子树最大贡献值（负贡献舍弃，取0）
9     const left = Math.max(dfs(node.left), 0)
10    // 右子树最大贡献值（负贡献舍弃，取0）
11    const right = Math.max(dfs(node.right), 0)
12
13    // 更新全局最大路径和
14    ans = Math.max(ans, node.val + left + right)
15    // 返回当前节点的最大贡献值（只能选左或右一条路径）
16    return node.val + Math.max(left, right)
17  }
18
19  dfs(root)
20  return ans
21 }
```

图论

岛屿数量

思路：深度优先搜索（DFS）或广度优先搜索（BFS），遍历网格，遇到陆地（'1'）时，计数加1，同时将当前陆地及相邻的陆地全部置为'0'（标记已访问），避免重复计数。

```
1 function numIslands(grid) {
2   const m = grid.length
3   const n = grid[0].length
4   let count = 0
5
6   // DFS: 标记当前陆地及相邻陆地为已访问
7   function dfs(i, j) {
8     if (i < 0 || i >= m || j < 0 || j >= n || grid[i][j] !== '1') return
9     grid[i][j] = '0' // 标记为已访问
10    // 遍历上下左右四个方向
11    dfs(i + 1, j)
12    dfs(i - 1, j)
13    dfs(i, j + 1)
14    dfs(i, j - 1)
15  }
16
17  // 遍历整个网格
18  for (let i = 0; i < m; i++) {
19    for (let j = 0; j < n; j++) {
20      if (grid[i][j] === '1') {
21        count++
22        dfs(i, j)
23      }
24    }
25  }
26
27  return count
28 }
```

腐烂的橘子

思路：广度优先搜索（BFS），先将所有腐烂的橘子（2）加入队列，记录新鲜橘子（1）的数量；每次遍历队列中的腐烂橘子，将其相邻的新鲜橘子腐烂，新鲜橘子数量减1；遍历结束后，若新鲜橘子数量为0，返回耗时，否则返回-1。

```
1 function orangesRotting(grid) {
2   const m = grid.length
3   const n = grid[0].length
4   const queue = []
5   let fresh = 0 // 新鲜橘子数量
6   let minutes = 0 // 耗时
```

```

7
8 // 初始化：将腐烂橘子加入队列，统计新鲜橘子数量
9 for (let i = 0; i < m; i++) {
10     for (let j = 0; j < n; j++) {
11         if (grid[i][j] === 2) queue.push([i, j])
12         if (grid[i][j] === 1) fresh++
13     }
14 }
15
16 const dirs = [[1,0],[-1,0],[0,1],[0,-1]] // 上下左右四个方向
17
18 // BFS遍历腐烂橘子
19 while (queue.length && fresh > 0) {
20     let size = queue.length
21     minutes++
22
23     for (let i = 0; i < size; i++) {
24         const [x, y] = queue.shift()
25         // 遍历四个方向
26         for (const [dx, dy] of dirs) {
27             const nx = x + dx
28             const ny = y + dy
29             // 检查相邻位置是否有新鲜橘子
30             if (nx >= 0 && nx < m && ny >= 0 && ny < n && grid[nx][ny] === 1) {
31                 grid[nx][ny] = 2 // 标记为腐烂
32                 fresh-- // 新鲜橘子数量减少
33                 queue.push([nx, ny]) // 加入队列，后续扩散
34             }
35         }
36     }
37 }
38
39 // 若还有新鲜橘子，返回-1，否则返回耗时
40 return fresh === 0 ? minutes : -1
41 }

```

课程表（拓扑排序）

思路（BFS版）

1. 统计每个课程的入度（前置课程数量）；2. 用队列存储入度为0的课程（可直接学习）；3. 遍历队列，每学习一门课程，减少其后续课程的入度，入度变为0则加入队列；4. 最终判断学习的课程数是否等于总课程数。

```

1 function canFinish(numCourses, prerequisites) {
2     // 入度数组
3     const inDegree = new Array(numCourses).fill(0);
4     // 邻接表（存储后续课程）

```

```

5   const adj = new Array(numCourses).fill(0).map(() => []);
6
7   // 初始化入度和邻接表
8   for (const [course, pre] of prerequisites) {
9       inDegree[course]++;
10      adj[pre].push(course);
11  }
12
13  const queue = [];
14  // 入度为0的课程入队
15  for (let i = 0; i < numCourses; i++) {
16      if (inDegree[i] === 0) queue.push(i);
17  }
18
19  let count = 0; // 已学习的课程数
20  while (queue.length) {
21      const cur = queue.shift();
22      count++;
23      // 减少后续课程的入度
24      for (const next of adj[cur]) {
25          inDegree[next]--;
26          if (inDegree[next] === 0) queue.push(next);
27      }
28  }
29
30  // 所有课程都能学习则返回true
31  return count === numCourses;
32 }

```

时间复杂度 $O(n + m)$ (n 为课程数, m 为先修课程对数), 空间复杂度 $O(n + m)$ 。

回溯

核心思路: 穷举所有可能, 通过“选/不选”或“选哪一个”的分支, 回溯时撤销选择, 剪枝优化效率。

全排列

思路: 用used数组标记已选择的元素, 递归选择未标记元素, 加入路径, 回溯时撤销标记, 直到路径长度等于数组长度。

```

1  function permute(nums) {
2      const res = [];
3      const path = [];
4      const used = new Array(nums.length).fill(false);
5
6      const backtrack = () => {
7          // 终止条件: 路径长度等于数组长度
8          if (path.length === nums.length) {
9              res.push([...path]);

```

```

10     return;
11 }
12 // 遍历所有元素，选择未使用的
13 for (let i = 0; i < nums.length; i++) {
14     if (used[i]) continue;
15     // 选择
16     used[i] = true;
17     path.push(nums[i]);
18     // 递归
19     backtrack();
20     // 回溯（撤销选择）
21     path.pop();
22     used[i] = false;
23 }
24 };
25
26 backtrack();
27 return res;
28 }

```

时间复杂度 $O(n * n!)$ ，空间复杂度 $O(n)$ （递归栈+used数组）。

子集

思路：子集不考虑顺序，每个元素有“选”和“不选”两种选择，递归时通过索引控制，避免重复。

```

1  function subsets(nums) {
2      const res = [];
3      const path = [];
4
5      const backtrack = (startIndex) => {
6          // 每次递归都记录当前路径（子集）
7          res.push([...path]);
8          // 从startIndex开始，避免重复选择
9          for (let i = startIndex; i < nums.length; i++) {
10             // 选择
11             path.push(nums[i]);
12             // 递归（下一个元素从i+1开始）
13             backtrack(i + 1);
14             // 回溯
15             path.pop();
16         }
17     };
18
19     backtrack(0);
20     return res;
21 }

```

时间复杂度 $O(n * 2^n)$ ，空间复杂度 $O(n)$ 。

括号生成

思路：核心约束：左括号数量 $\leq n$ ，右括号数量 $\leq$ 左括号数量。递归生成，满足约束则继续，达到 $2n$ 长度则记录结果。

```
1 function generateParenthesis(n) {
2   const res = [];
3   const path = [];
4
5   const backtrack = (left, right) => {
6     // 终止条件：括号总数为2n
7     if (path.length === 2 * n) {
8       res.push(path.join(''));
9       return;
10    }
11    // 左括号数量不足n，可添加左括号
12    if (left < n) {
13      path.push('(');
14      backtrack(left + 1, right);
15      path.pop();
16    }
17    // 右括号数量小于左括号，可添加右括号
18    if (right < left) {
19      path.push(')');
20      backtrack(left, right + 1);
21      path.pop();
22    }
23  };
24
25  backtrack(0, 0);
26  return res;
27 }
```

时间复杂度 $O(4^n / \sqrt{n})$ （卡特兰数），空间复杂度 $O(n)$ （递归栈）。

N 皇后

思路：每行放一个皇后，用三个集合分别标记列、左对角线（行-列固定）、右对角线（行+列固定），避免皇后互相攻击，回溯放置每一行的皇后。

```
1 function solveNQueens(n) {
2   const res = [];
3   // 标记列、左对角线 (row - col)、右对角线 (row + col)
4   const col = new Set();
5   const diag1 = new Set();
6   const diag2 = new Set();
7 }
```

```

8   const backtrack = (row, path) => {
9       if (row === n) {
10          // 格式化结果（将路径转为棋盘字符串）
11          const board = path.map(i => '.'.repeat(i) + 'Q' + '.'.repeat(n - i -
12          1));
13          res.push(board);
14          return;
15      }
16      for (let i = 0; i < n; i++) {
17          // 跳过被攻击的位置
18          if (col.has(i) || diag1.has(row - i) || diag2.has(row + i)) continue;
19          // 选择
20          col.add(i);
21          diag1.add(row - i);
22          diag2.add(row + i);
23          path.push(i);
24          // 递归下一行
25          backtrack(row + 1, path);
26          // 回溯
27          path.pop();
28          diag2.delete(row + i);
29          diag1.delete(row - i);
30          col.delete(i);
31      }
32  };
33  backtrack(0, []);
34  return res;
35  }

```

时间复杂度 $O(n!)$ ，空间复杂度 $O(n)$ （递归栈+标记集合）。

动态规划模板

核心思路：将大问题拆分为小问题，存储小问题的解（dp数组），通过状态转移方程推导大问题的解，避免重复计算。

最大子数组和

思路：dp[i] 表示以第i个元素结尾的最大子数组和；状态转移：dp[i] = max(nums[i], dp[i-1] + nums[i])；最终取dp数组的最大值。

```

1 function maxSubArray(nums) {
2   const n = nums.length;
3   // dp数组优化：可只用一个变量存储前一个状态，节省空间
4   let pre = nums[0];
5   let max = pre;
6
7   for (let i = 1; i < n; i++) {
8     pre = Math.max(nums[i], pre + nums[i]);
9     max = Math.max(max, pre);
10  }
11
12  return max;
13 }

```

时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ （优化后）。

打家劫舍

思路：dp[i] 表示前i间房子能偷到的最大金额；状态转移：dp[i] = max(dp[i-1]（不偷第i间），dp[i-2] + nums[i-1]（偷第i间））。

```

1 function rob(nums) {
2   const n = nums.length;
3   if (n === 0) return 0;
4   if (n === 1) return nums[0];
5   // 优化空间：用两个变量存储dp[i-1]和dp[i-2]
6   let prev2 = nums[0];
7   let prev1 = Math.max(nums[0], nums[1]);
8
9   for (let i = 2; i < n; i++) {
10    const curr = Math.max(prev1, prev2 + nums[i]);
11    prev2 = prev1;
12    prev1 = curr;
13  }
14
15  return prev1;
16 }

```

时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ （优化后）。

零钱兑换

思路：dp[i] 表示凑成金额i所需的最少硬币数；初始化dp[0] = 0，其余为无穷大；状态转移：dp[i] = min(dp[i], dp[i - coin] + 1)（遍历所有硬币）。

```

1 function coinChange(coins, amount) {
2   // 初始化dp数组，长度为amount+1，初始值为无穷大
3   const dp = new Array(amount + 1).fill(Infinity);
4   dp[0] = 0; // 凑0元需要0枚硬币

```

```

5
6   for (let i = 1; i <= amount; i++) {
7       // 遍历所有硬币，尝试用当前硬币凑金额i
8       for (const coin of coins) {
9           if (i >= coin) {
10              dp[i] = Math.min(dp[i], dp[i - coin] + 1);
11          }
12      }
13  }
14
15  // 若dp[amount]仍为无穷大，说明无法凑成
16  return dp[amount] === Infinity ? -1 : dp[amount];
17 }

```

时间复杂度 $O(\text{amount} * k)$ (k 为硬币种类数)，空间复杂度 $O(\text{amount})$ 。

单词拆分

思路：dp[i] 表示前i个字符组成的字符串能否被拆分；状态转移：若存在j < i，使得dp[j]为true且s[j..i)在单词表中，则dp[i]为true。

```

1  function wordBreak(s, wordDict) {
2      const n = s.length;
3      const dp = new Array(n + 1).fill(false);
4      dp[0] = true; // 空字符串可拆分
5      const wordSet = new Set(wordDict);
6
7      for (let i = 1; i <= n; i++) {
8          // 遍历所有可能的拆分点j
9          for (let j = 0; j < i; j++) {
10             if (dp[j] && wordSet.has(s.slice(j, i))) {
11                 dp[i] = true;
12                 break; // 找到一个拆分方式即可
13             }
14         }
15     }
16
17     return dp[n];
18 }

```

时间复杂度 $O(n^2)$ ，空间复杂度 $O(n + k)$ (k 为单词表长度)。

最长递增子序列

思路：dp[i] 表示以第i个元素结尾的最长递增子序列长度；状态转移：遍历j < i，若nums[j] < nums[i]，则dp[i] = max(dp[i], dp[j] + 1)；最终取dp数组最大值。

```

1  function lengthOfLIS(nums) {
2      const n = nums.length;

```

```

3   if (n === 0) return 0;
4   const dp = new Array(n).fill(1); // 每个元素自身是长度为1的子序列
5   let maxLen = 1;
6
7   for (let i = 1; i < n; i++) {
8     for (let j = 0; j < i; j++) {
9       if (nums[j] < nums[i]) {
10        dp[i] = Math.max(dp[i], dp[j] + 1);
11      }
12    }
13    maxLen = Math.max(maxLen, dp[i]);
14  }
15
16  return maxLen;
17 }

```

时间复杂度 $O(n^2)$ ，空间复杂度 $O(n)$ 。

编辑距离

思路：dp[i][j] 表示将word1前i个字符转为word2前j个字符的最少操作数；状态转移：若 word1[i-1] === word2[j-1]，则dp[i][j] = dp[i-1][j-1]；否则取“插入、删除、替换”三者最小值+1。

```

1  function minDistance(word1, word2) {
2    const m = word1.length;
3    const n = word2.length;
4    // 初始化dp数组，dp[i][j]对应word1前i个、word2前j个字符
5    const dp = new Array(m + 1).fill(0).map(() => new Array(n + 1).fill(0));
6
7    // 边界：word2为空，需删除word1所有字符
8    for (let i = 0; i <= m; i++) {
9      dp[i][0] = i;
10   }
11   // 边界：word1为空，需插入word2所有字符
12   for (let j = 0; j <= n; j++) {
13     dp[0][j] = j;
14   }
15
16   for (let i = 1; i <= m; i++) {
17     for (let j = 1; j <= n; j++) {
18       if (word1[i - 1] === word2[j - 1]) {
19         dp[i][j] = dp[i - 1][j - 1];
20       } else {
21         // 插入 (dp[i][j-1]+1)、删除 (dp[i-1][j]+1)、替换 (dp[i-1][j-1]+1)
22         dp[i][j] = Math.min(dp[i][j - 1], dp[i - 1][j], dp[i - 1][j - 1]) + 1;

```

```
23     }
24     }
25     }
26
27     return dp[m][n];
28 }
```

时间复杂度 $O(m * n)$ ，空间复杂度 $O(m * n)$ （可优化为 $O(\min(m,n))$ ）。